

HTML + JavaScript CRUD Tasks — Complete Answers

This PDF answers all 13 requested tasks. Repeated tasks are covered by the same implementation. The examples use plain HTML, CSS, and JavaScript and store data in in-memory arrays.

1, 2, 4, 5, 7, 8 — Spotify-style Playlist Manager

The form collects a playlist name and description. JavaScript stores each playlist in the **playlists** array, renders the list below the form, and supports Edit and Delete. The display is re-rendered after every add, edit, or delete.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Playlist Manager</title>
<style>
body{font-family:Arial;max-width:700px;margin:40px auto}
form{display:grid;gap:10px}
input,textarea,button{padding:10px;font-size:16px}
.playlist{border:1px solid #ddd;padding:12px;margin-top:10px}
.actions button{margin-right:6px;font-size:14px}
</style>
</head>
<body>
<h1>Create Playlist</h1>
<form id="playlistForm">
<input id="playlistName" placeholder="Playlist name" required>
<textarea id="playlistDescription" placeholder="Description" required></textarea>
<button type="submit">Save Playlist</button>
</form>
<h2>My Playlists</h2>
<div id="playlistList"></div>

<script>
let playlists = [];
let editingIndex = -1;

const form = document.getElementById("playlistForm");
const nameInput = document.getElementById("playlistName");
const descInput = document.getElementById("playlistDescription");
const list = document.getElementById("playlistList");

form.addEventListener("submit", function(event) {
  event.preventDefault();

  const playlist = {
    name: nameInput.value.trim(),
    description: descInput.value.trim()
  };

  if (editingIndex === -1) {
    playlists.push(playlist);
  } else {
    playlists[editingIndex] = playlist;
    editingIndex = -1;
  }

  form.reset();
  renderPlaylists();
});

function renderPlaylists() {
  list.innerHTML = "";

  playlists.forEach((playlist, index) => {
    const card = document.createElement("div");
    card.className = "playlist";
    card.innerHTML = `
    <h3>${escapeHtml(playlist.name)}</h3>
    <p>${escapeHtml(playlist.description)}</p>
    <div class="actions">
      <button onclick="editPlaylist(${index})">Edit</button>
      <button onclick="deletePlaylist(${index})">Delete</button>
    </div>`;
    list.appendChild(card);
  });
}

function editPlaylist(index) {
  nameInput.value = playlists[index].name;
  descInput.value = playlists[index].description;
  editingIndex = index;
}
```

```
    nameInput.focus();
  }

function deletePlaylist(index) {
  playlists.splice(index, 1);
  if (editingIndex === index) {
    editingIndex = -1;
    form.reset();
  }
  renderPlaylists();
}

function escapeHtml(value) {
  const div = document.createElement("div");
  div.textContent = value;
  return div.innerHTML;
}
</script>
</body>
</html>
```

3 — Flipkart-style Wishlist Manager Architecture

The same CRUD architecture can be used for a Flipkart-style wishlist. The storage layer can be an array for a simple exercise or localStorage when data should survive page refreshes.

USER	FORM / UI	JAVASCRIPT	STORAGE	DISPLAY LIST
Enter item	Add Item Submit	Read form data Validate	Array / localStorage Save item	Render items
Click Edit	Edit form	Find item Update object	Save updated data	Re-render UI
Click Delete	Delete button	splice() / filter()	Save updated data	Re-render UI

Flow: Add Item Form → Form Submit → Validate/Collect Data → Add to Array or localStorage → Update Display.
Edit/Delete actions → Update Storage → Re-render Display.

6, 9, 12, 13 — Zomato-style Restaurant Manager

The form collects restaurant name, cuisine, and rating. Each entry is stored in the **restaurants** array. The list shows all three fields with Edit and Delete buttons. Delete uses **splice()**, and Edit pre-fills the form and saves the updated object.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Restaurant Manager</title>
<style>
body{font-family:Arial;max-width:700px;margin:40px auto}
form{display:grid;gap:10px}
input,button{padding:10px;font-size:16px}
.restaurant{border:1px solid #ddd;padding:12px;margin-top:10px}
.actions button{margin-right:6px;font-size:14px}
</style>
</head>
<body>
<h1>Add Restaurant</h1>
<form id="restaurantForm">
<input id="restaurantName" placeholder="Restaurant name" required>
<input id="cuisine" placeholder="Cuisine" required>
<input id="rating" type="number" min="0" max="5" step="0.1"
placeholder="Rating (0-5)" required>
<button type="submit">Save Restaurant</button>
</form>
<h2>Restaurants</h2>
<div id="restaurantList"></div>

<script>
let restaurants = [];
let editingIndex = -1;

const form = document.getElementById("restaurantForm");
const nameInput = document.getElementById("restaurantName");
const cuisineInput = document.getElementById("cuisine");
const ratingInput = document.getElementById("rating");
const list = document.getElementById("restaurantList");

form.addEventListener("submit", function(event) {
  event.preventDefault();

  const restaurant = {
    name: nameInput.value.trim(),
    cuisine: cuisineInput.value.trim(),
    rating: Number(ratingInput.value)
  };

  if (editingIndex === -1) {
    restaurants.push(restaurant);
  } else {
    restaurants[editingIndex] = restaurant;
    editingIndex = -1;
  }

  form.reset();
  renderRestaurants();
});
```

```

function renderRestaurants() {
  list.innerHTML = "";

  restaurants.forEach((restaurant, index) => {
    const card = document.createElement("div");
    card.className = "restaurant";
    card.innerHTML = `
      <h3>${escapeHtml(restaurant.name)}</h3>
      <p>Cuisine: ${escapeHtml(restaurant.cuisine)}</p>
      <p>Rating: ${restaurant.rating}/5</p>
      <div class="actions">
        <button onclick="editRestaurant(${index})">Edit</button>
        <button onclick="deleteRestaurant(${index})">Delete</button>
      </div>`;
    list.appendChild(card);
  });
}

function editRestaurant(index) {
  nameInput.value = restaurants[index].name;
  cuisineInput.value = restaurants[index].cuisine;
  ratingInput.value = restaurants[index].rating;
  editingIndex = index;
  nameInput.focus();
}

function deleteRestaurant(index) {
  restaurants.splice(index, 1);
  if (editingIndex === index) {
    editingIndex = -1;
    form.reset();
  }
  renderRestaurants();
}

function escapeHtml(value) {
  const div = document.createElement("div");
  div.textContent = value;
  return div.innerHTML;
}
</script>
</body>
</html>

```

10, 11 — Playlist Data-Flow Diagram

Form Description	→	Form Submit preventDefault()	→	Add to Array playlists.push()	→	Update Display renderPlaylists()
Button	→	Pre-fill Form select index	→	Update Array playlists[index] = ...	→	Update Display
Button	→	Find Index	→	Delete from Array splice(index, 1)	→	Update Display

Task-by-task answer summary

Task	Answer / Implementation
1–2	Playlist form + JavaScript array storage implemented.
3	Wishlist architecture diagram shows form → storage → display and CRUD flow.
4–5	Playlist list renders name and description after every add.
6	Restaurant form collects name, cuisine, rating into an array.
7–8	Playlist Edit/Delete buttons update the array and UI instantly.
9	Restaurant list displays name, cuisine, rating with Edit/Delete.
10–11	Playlist data-flow diagram included with labeled arrows.
12	Restaurant Delete uses splice() and re-renders immediately.
13	Restaurant Edit pre-fills the form and updates the array/UI.

How to run: Copy each complete code block into its own .html file, for example *playlist-manager.html* and *restaurant-manager.html*, then open the files in a browser.